Few-Shot Prompting & Neighboring Tabs Context

Question: A developer is writing a custom data-serialization parser for a proprietary legacy format in `parser.py`. When they write a comment describing the function, GitHub Copilot generates generic JSON parsing code instead of matching the proprietary delimiter structure. Which prompt-engineering technique most reliably steers Copilot to follow the exact custom format?

A. Closing all other open editor tabs so Copilot has zero surrounding file context.

B. Providing 2–3 concrete input-to-output examples (few-shot prompting) in comments or keeping an existing parser implementation open in a neighboring tab. **[CORRECT ANSWER]**

C. Writing a single vague one-word comment such as `# parse` above the function signature.

D. Switching to `@vscode` in the Chat view to ask for Python syntax documentation.

Explanation & Exam Rationale

Large language models excel at in-context pattern continuation. When working with proprietary schemas or domain-specific conventions not widely represented in public training data, **few-shot prompting** (supplying 2–3 input/output pairs directly in comments or tests) and **neighboring tab context** (opening sibling files with matching idioms) give Copilot concrete patterns to replicate.

Prompting Strategy	Definition	Best Used When
Zero-Shot	Task description without explicit examples	Standard algorithms, well-known libraries, boilerplate CRUD
Few-Shot	Including 2–3 concrete input/output examples in prompt/comments	Custom data formats, edge-case transformations, strict output schemas
Chain-of-Thought / Stepwise	Breaking a complex task into sequential sub-steps	Multi-stage business logic, algorithmic pipelines, refactoring
Open Neighboring Tabs	Keeping relevant interfaces, models, or sibling files open in IDE	Ensuring inline completions match project types, imports, and style

Key Exam Takeaway: Copilot automatically harvests context from **currently open tabs** in your IDE (especially recently viewed files of the same language), making tab hygiene + few-shot examples two of the highest-leverage prompt engineering practices.